

DevHire — Internship Project Plan

Features, role assignments, and 2-week schedule for 3 CSE interns

Project	DevHire — Full-Stack Job Board Platform
Stack	Vue 3 + Vite (Frontend) · Node.js + Express + MongoDB (Backend)
Duration	2 weeks · 10 working days per intern
Interns	3 (Intern A — Frontend, Intern B — Backend, Intern C — Full Stack/QA)
Deliverable	Live deployed app + GitHub repo + README + 10-min demo

SECTION 1 — PROJECT FEATURES & OWNERSHIP

All 9 features of the DevHire platform, with who owns each one.

1. Authentication system

Backend | Frontend

JWT register/login for two roles (employer and seeker). Token stored in localStorage, auto-injected via Axios interceptors. Role-based route guards on Vue Router.

2. Job listings & CRUD

Backend | Frontend

Employers create, edit, delete jobs. Seekers browse paginated job cards. Full MongoDB text index for full-text search. Soft-delete via isActive flag.

3. Search & filter

Intern C

Debounced full-text search via useSearch composable. Filter by job type and location with query params. Results update without page reload.

4. Apply with resume

Backend | Frontend

Seekers upload PDF via Multer (5 MB cap, PDF only). One application per job enforced at DB level with compound unique index. ApplyModal component with client-side file validation.

5. Employer dashboard

Backend | Frontend

Lists all posted jobs with applicant count. View applicants per job with resume download links. Update application status: pending / shortlisted / rejected.

6. Seeker dashboard	Frontend
Application history with live status badges. Shows job title, company and location. Linked back to original job listings.	
7. Testing suite	Intern C
Jest + Supertest for backend routes. Vitest + Vue Test Utils for components. Minimum 5 BE tests and 3 FE tests required before deployment.	
8. Deployment	Intern C
Backend on Render with env vars (MONGO_URI, JWT_SECRET). Frontend on Vercel pointing to live API URL. Live URL is a required deliverable for demo day.	
9. Documentation	All three
README with setup guide, full API route table, env var reference, and screenshots. Required final deliverable for all three interns.	

SECTION 2 — ROLE ASSIGNMENTS & RESPONSIBILITIES

A

Intern A — Frontend engineer

Vue 3 · Pinia · Vue Router · Axios · Vite

- Own all .vue pages and components — NavBar, JobCard, ApplyModal, Login, Register, Jobs, JobDetail, seeker dashboard, employer dashboard
- Implement Pinia stores (auth.js, jobs.js) and the Axios API service with interceptors
- Set up Vue Router with protected routes and role-based guards (seeker vs employer)
- Build the employer post-job form and seeker application history view
- Ensure all pages are fully responsive across mobile and desktop viewports
- Write 3 Vitest component tests by end of Week 2

B

Intern B — Backend engineer

Node.js · Express · MongoDB · Mongoose · JWT · Multer

- Own the entire Node.js/Express API — all routes, controllers, models, and middleware
- Design and implement the MongoDB schema for Users, Jobs, and Applications with proper indexes
- Implement JWT-based authentication with bcrypt password hashing (12 salt rounds)
- Build the file upload system with Multer — PDF-only filter, 5 MB size cap, diskStorage
- Write clean RESTful API with consistent HTTP status codes and error message format
- Provide API reference document (routes, request body, response shape) to Intern A by end of Week 1

C

Intern C — Full stack / QA / DevOps

Search feature · Jest/Vitest · Render · Vercel · Documentation

- Own the search & filter feature end-to-end: backend query params and frontend useSearch composable
- Write the full test suite — Jest/Supertest (BE) and Vitest + Vue Test Utils (FE)
- Handle all deployment: Render (backend) and Vercel (frontend), env vars, live URL verification
- Perform integration testing across the full stack — catch contract gaps between FE and BE
- Write the complete project README — setup guide, API table, env vars, screenshots
- Prepare and lead the final 10-minute demo presentation on Friday of Week 2

SECTION 3 — 2-WEEK DAY-BY-DAY PLANS

Intern A — Frontend engineer

Vue 3 · Pinia · Vue Router · Axios · Vite

Week 1 — Foundation

Mon	■ Set up Vite + Vue 3 project, install Pinia and Vue Router ■ Create full folder structure: pages/, components/, stores/, services/, composables/ ■ Write global CSS — variables, .btn, .card, .badge, .form-group utility classes
Tue	■ Set up Axios service (api.js) with base URL and auth interceptor ■ Build auth Pinia store — login(), register(), logout() with localStorage persistence ■ Build Login.vue and Register.vue pages with form validation and error display
Wed	■ Configure Vue Router — all routes, beforeEach guard, role-based redirect ■ Build NavBar.vue with dynamic links based on auth state and user role ■ Build Home.vue landing page with hero and CTA buttons
Thu	■ Build jobs Pinia store — fetchJobs(), fetchJob(), createJob(), deleteJob() ■ Build Jobs.vue — sidebar filter layout, renders JobCard list from store ■ Build JobCard.vue — title, company, location badge, skill chips, hover effect
Fri	■ Build JobDetail.vue — full job info, skills list, Apply Now button for seekers ■ Test Login → Browse → JobDetail flow end-to-end with backend running ■ Week 1 standup demo — show auth and job listing to mentor

Milestone: Login, register, job listing, and job detail all working with live API

Week 2 — Polish & Complete

Mon	■ Build ApplyModal.vue — file input, cover note textarea, multipart POST to API ■ Build seeker MyApplications.vue — fetch history, status badges (pending/shortlisted/rejected) ■ Build employer PostJob.vue — form with all job fields, comma-separated skill tag input
Tue	■ Build employer Dashboard.vue — job list with view and delete per row ■ Employer job detail: show applicant list with resume link and status dropdown ■ Wire all new pages to router, verify guards work correctly for both roles
Wed	■ Responsive design pass — fix breakpoints on Jobs sidebar, Dashboard, and modals ■ Add empty states and loading spinners on all async pages ■ Graceful error handling — show user-friendly messages on all API failures
Thu	■ Write 3 Vitest tests: NavBar renders correctly, JobCard displays props, Login form validation ■ Cross-browser check (Chrome, Firefox, Safari) and fix any layout issues ■ Set VITE_API_URL to deployed backend URL, verify production build works

Fri

- Take screenshots of all key pages for README and portfolio use
- Prepare 5-min demo walkthrough: register as employer, post job, apply as seeker
- Final demo presentation and team retrospective

Milestone: All pages complete, responsive, tested, and connected to live deployed backend

Intern B — Backend engineer

Node.js · Express · MongoDB · Mongoose · JWT · Multer

Week 1 — Build the full API

Mon	■ Initialise Node.js project, install Express, Mongoose, dotenv, cors, bcryptjs, jsonwebtoken, multer ■ Set up folder structure: config/, models/, controllers/, middleware/, routes/, uploads/ ■ Write db.js MongoDB connection with error handling and process.exit on failure
Tue	■ Build User model — name, email, password (pre-save hash), role enum, company field ■ Build auth controller — register (duplicate check, hash, return JWT), login (compare + JWT) ■ Build protect() middleware and requireRole() higher-order function for route guards
Wed	■ Build Job model with text index on title, company, description ■ Build jobs controller — getJobs (text search + pagination), getJob, createJob, updateJob, soft-deleteJob, getMyJobs ■ Wire auth and job routes, test all job endpoints in Postman
Thu	■ Build Application model with compound unique index on job + applicant ■ Build Multer upload middleware — diskStorage, PDF-only fileFilter, 5 MB limit ■ Build applications controller — apply (duplicate guard), getApplicants, updateStatus, myApplications
Fri	■ Wire application routes with correct middleware chain (protect + requireRole + upload) ■ Test all endpoints in Postman: auth flow, CRUD, file upload, error cases ■ Write and share API reference doc (route table, request/response shapes) with Intern A

Milestone: All API routes working locally — auth, jobs CRUD, file upload, and applications

Week 2 — Harden & Support

Mon	■ Add search query params to GET /jobs — q (text), type, location, page, limit ■ Pair with Intern A to integrate frontend auth — test register/login end-to-end ■ Add input validation — required fields, email format, password length, enum values
Tue	■ Support frontend job listing integration — verify CORS, response shape, and pagination ■ Improve error responses — consistent { message } format, correct HTTP codes (400/401/403/404/500) ■ Verify that employer-only routes correctly reject seeker tokens (and vice versa)
Wed	■ Write Jest + Supertest tests: auth register/login/me, job CRUD, application flow ■ Support Intern A with ApplyModal integration — multipart/form-data handling ■ Performance: add .lean() to read-only Mongoose queries; review and confirm indexes
Thu	■ Prepare production env — set NODE_ENV=production, review CORS allowed origins ■ Assist Intern C with Render deployment — environment variables, start command ■ Fix any bugs found during frontend integration testing

Fri

- Final backend test run — confirm all routes pass with no regressions
- Write backend section of README — setup steps, env vars, API route table
- Demo: walk through API in Postman for mentor, explain auth flow and file upload

Milestone: Hardened API live on Render, all tests passing, frontend fully integrated

Intern C — Full stack / QA / DevOps

Search feature · Jest/Vitest · Render · Vercel · Documentation

Week 1 — Feature & Test Setup

Mon	■ Set up both frontend and backend repos locally and confirm both servers run ■ Install Jest + Supertest in backend; install Vitest + Vue Test Utils in frontend ■ Create test config files (jest.config.js, vitest.config.js) and placeholder test files
Tue	■ Build useSearch.js composable — reactive query/type/location refs with 350 ms debounce ■ Extend GET /jobs backend to handle ?q=, ?type=, ?location= query params ■ Manually test search with Postman — verify text index hits, location regex, type filter
Wed	■ Integrate useSearch into Jobs.vue — sidebar inputs drive debounced API calls ■ Write first 2 BE tests: POST /register rejects missing fields, POST /login rejects bad credentials ■ Write first FE test: NavBar shows correct links based on auth state
Thu	■ Write 3 more BE tests: GET /me unauthenticated → 401, POST /jobs as seeker → 403, GET /jobs returns array ■ Write 2 more FE tests: JobCard renders title/company, Login form shows error on bad submit ■ File issues for any bugs found during testing — coordinate fixes with Interns A and B
Fri	■ Run full test suite — confirm all tests pass on a clean install ■ Full end-to-end integration check: register → post job → search → apply → view applicants ■ Week 1 standup — report test coverage, bugs found, and search feature status

Milestone: Search working, 8 tests passing, full flow verified end-to-end

Week 2 — Deploy, Document & Demo

Mon	■ Deploy backend to Render — connect GitHub repo, set MONGO_URI and JWT_SECRET env vars ■ Smoke-test live backend with Postman — verify all routes respond on production URL ■ Update frontend .env with VITE_API_URL pointing to Render URL
Tue	■ Deploy frontend to Vercel — connect repo, set env var, trigger production build ■ Full smoke test on live Vercel URL: auth, job CRUD, apply, dashboard, search ■ Fix any production-specific bugs (CORS, env vars, static file paths)
Wed	■ Mobile QA on live URL — test all core flows on a real phone ■ Add edge-case tests: apply twice to same job → 400, employer accessing seeker route → 403 ■ Start writing README — project overview, tech stack table, setup steps
Thu	■ Complete README — API route table, env var reference, deployment guide, screenshots ■ Prepare demo script: employer registers → posts job → seeker registers → searches → applies → employer shortlists ■ Final test run on both local and live — confirm zero regressions

Fri

- Lead the 10-minute final demo for mentor
- Present test coverage summary and deployment architecture
- Team retrospective — what went well, what to improve

Milestone: Live URL, full test suite, complete README, successful 10-min demo delivered